

Lab 8: Implement Kruskal's Algorithm for MST

Theory:

Kruskal's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, undirected graph. It sorts all the edges in non-decreasing order of their weights and picks the smallest edge. It includes the edge in the MST if it doesn't form a cycle with the already included edges. The process continues until $(V - 1)$ edges have been selected, forming a spanning tree that connects all V vertices with minimum total weight.

To efficiently detect cycles, Kruskal's algorithm uses the Union-Find (Disjoint Set Union) data structure. Each vertex starts as its own set. When an edge (u, v) is considered, the FIND operation checks whether u and v belong to the same component. If they don't, the edge is safe to include and the UNION operation merges the two sets. With path compression, these operations run in nearly $O(1)$ amortised time.

Expressing Complexity of cases

Case	Time Complexity	Condition
Best	$O(E \log E)$ or $O(E \log V)$	Sparse graph
Average	$O(E \log E)$ or $O(E \log V)$	General graph
Worst	$O(E \log E)$ or $O(E \log V)$	Dense graph, $E \sim V^2$

Algorithm

1. Sort all edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far.
3. If cycle is not formed, include this edge. Else, discard it.
4. Repeat step 2 until there are $(V-1)$ edges in the spanning tree.

Pseudocode

BEGIN

 SORT edges in non-decreasing order of weight

 CREATE empty $MST = \{\}$

 FOR each vertex v in Graph DO

 MAKE-SET(v)

 END FOR

 FOR each edge (u, v) in sorted edge list DO

 IF FIND-SET(u) $\neq$ FIND-SET(v) THEN

 ADD edge (u, v) to MST

 UNION(u, v)

 END IF

 END FOR

RETURN MST

END

Conclusion:

The experiment successfully demonstrates Kruskal's algorithm finding the Minimum Spanning Tree of a weighted undirected graph. The algorithm sorted all five edges by weight and selected three edges — $2 \rightarrow 3$ (cost 4), $0 \rightarrow 3$ (cost 5), and $0 \rightarrow 1$ (cost 10) — yielding a total minimum cost of 19, connecting all four vertices without forming any cycle.

The use of the Union-Find data structure with path compression ensures that cycle detection is performed in near-constant amortised time, making the overall algorithm highly efficient. The uniform time complexity of $O(E \log E)$ across all cases makes Kruskal's approach particularly well-suited for sparse graphs where the number of edges is close to the number of vertices. The algorithm has practical applications in network design, road planning, and cluster analysis.